

BG Bank RPC Project

Secure Banking Web Application

Team 3

Institute of Technology of Cambodia
Department of Applied Mathematics and Statistics

Introduction to Parallel and Distributed Systems
I4-AMS-DAS-A (2025–2026)

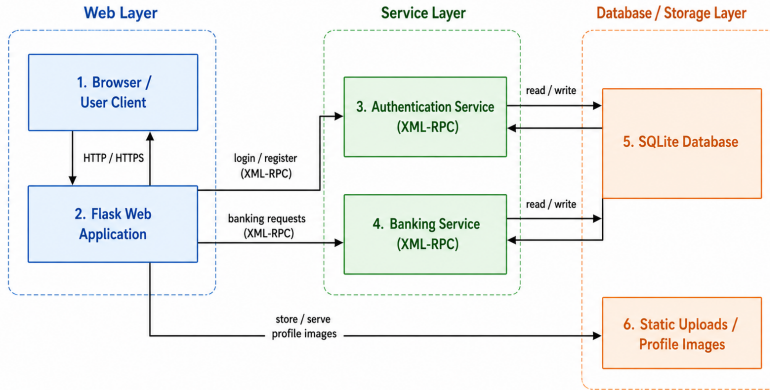
Project Overview

- BG Bank is a secure banking web application built with Flask and XML-RPC.
- The system separates the web interface, authentication service, banking service, and database.
- Users can register, log in, view a dashboard, deposit, withdraw, transfer, view history, and update profile information.
- The project demonstrates distributed service communication using Remote Procedure Call.

Project Objectives

- Build a complete banking web application for customer account operations.
- Use XML-RPC for communication between the web app and backend services.
- Protect user sessions with encrypted login tickets.
- Require an action password for sensitive transactions.
- Store users, profiles, accounts, transactions, and audit logs in SQLite.

SecureBG Banking System Architecture



Main Components

Component	Responsibility
Browser	Displays pages and sends user requests to Flask.
Flask Web App	Handles routes, forms, sessions, templates, and RPC calls.
Authentication Service	Handles registration, login, password hashing, and ticket issuing.
Banking Service	Handles balance, deposit, withdraw, transfer, history, dashboard, and profile operations.
SQLite Database	Stores users, accounts, profiles, transactions, and audit logs.

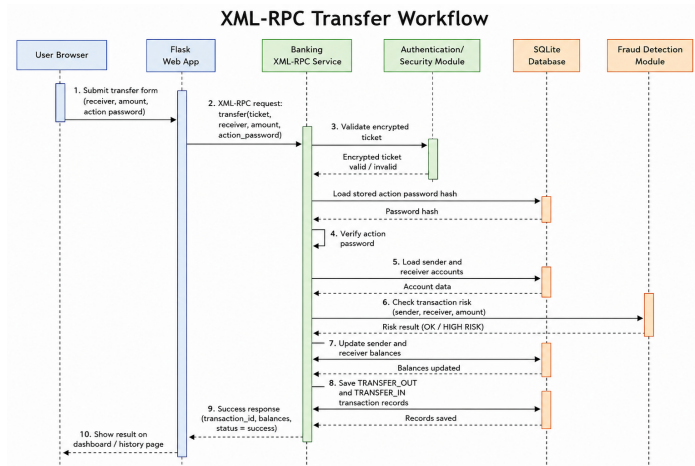
Authentication RPC

- `register_user()`
- `login()`
- Issues encrypted login tickets

Banking RPC

- `deposit()`
- `withdraw()`
- `transfer()`
- `view_transaction_history()`
- `get_dashboard_summary()`

Transfer Workflow

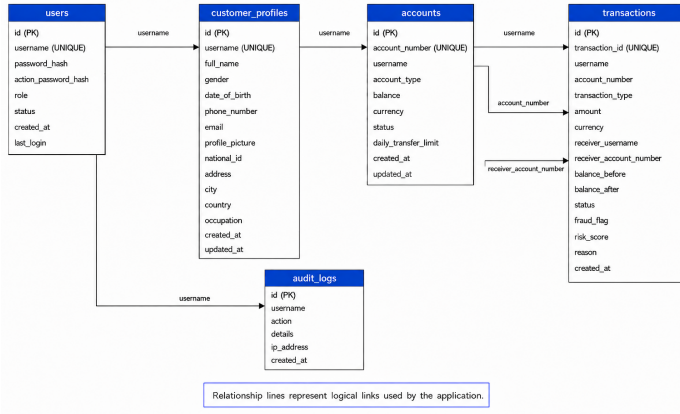


Security Features

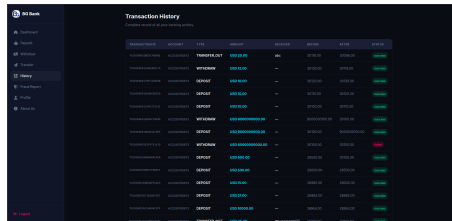
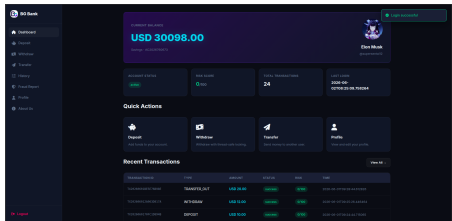
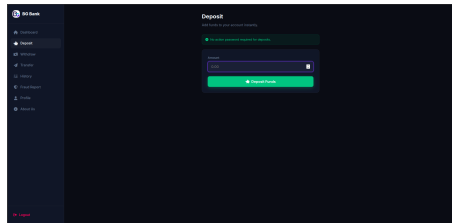
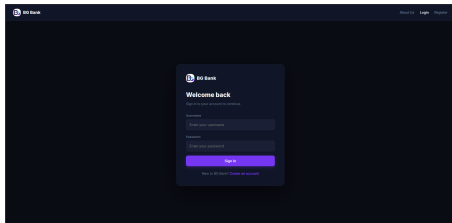
- Login passwords and action passwords are stored as hashes.
- Login creates a time-limited Fernet encrypted ticket.
- Banking RPC methods validate the ticket before processing requests.
- Withdraw and transfer require the action password.
- Audit logs record important account actions.

Database Design

SecureBG Banking Logical Database Schema



Application Screenshots



Docker Demonstration

The screenshot displays the Docker Desktop interface for a container named 'secure_bg_banking_web'. The container's path is '/media/super/DualSuper/ClassxV1/2Project IA/Introd2Prills/BankingSystem/secure_bg_banking_web'. The logs are organized into three sections: 'web', 'bank', and 'auth'. The 'web' section shows the Flask application starting, with a warning that it is a development server. The 'bank' section shows the database initialization and the banking server running. The 'auth' section shows the authentication service running.

```
secure_bg_banking_web
/media/super/DualSuper/ClassxV1/2Project IA/Introd2Prills/BankingSystem/secure_bg_banking_web

View configurations

web
secure_bg_banking
5000-5000
auth
2026-06-02 08:24:08,488 [DB] db_connection_open path=/data/banking.db
2026-06-02 08:24:08,490 [DB] database_initialized path=/data/banking.db
2026-06-02 08:24:08,490 [AUTH_RPC] service_start url=http://0.0.0.0:8001
Authentication Service running on http://0.0.0.0:8001

bank
secure_bg_banking
2026-06-02 08:24:08,608 [DB] db_connection_open path=/data/banking.db
2026-06-02 08:24:08,620 [DB] database_initialized path=/data/banking.db
2026-06-02 08:24:08,620 [BANK_RPC] service_start url=http://0.0.0.0:8002
Banking Server running on http://0.0.0.0:8002

auth
secure_bg_banking
* Serving Flask app 'web_app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.18.0.4:5000
Press CTRL+C to quit
```

Limitations and Future Work

- Use database transactions for registration and money operations.
- Store money as integer cents or Decimal values instead of floating point values.
- Add CSRF protection and stronger authentication controls.
- Add automated tests for registration, login, deposit, withdraw, and transfer.
- Use HTTPS and production-safe logging for deployment.

- BG Bank demonstrates a distributed banking workflow using Flask and XML-RPC.
- The project separates authentication, banking logic, security helpers, and database storage.
- Encrypted tickets and action passwords improve protection for account operations.
- The system is suitable for demonstrating RPC communication and service coordination.

Thank You